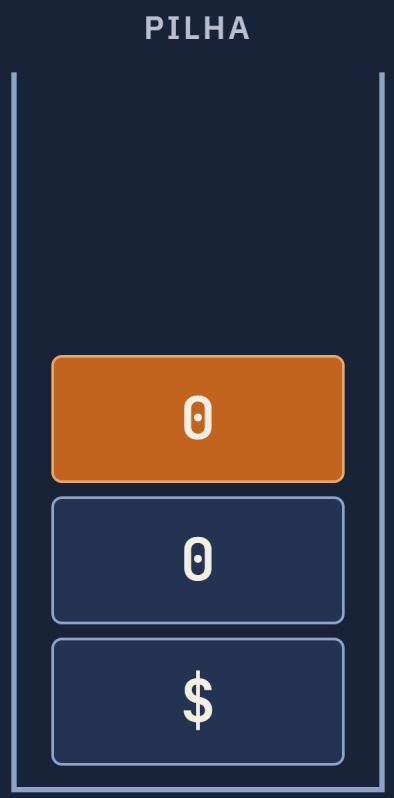
TEORIA DA COMPUTAÇÃO · CAPÍTULO 2 — LINGUAGENS LIVRES-DO-CONTEXTO

Autômatos com Pilha

Seção 2.2 de Sipser, *Introdução à Teoria da Computação* (2ª ed.): o modelo, a definição formal e três exemplos com rastreio da pilha.



Prof. Renê Rodrigues Veloso

UNIMONTES · Sistemas de Informação

ROTEIRO DA AULA

Cinco passos até o autômato com pilha

1	**Motivação**	O que o autômato finito não consegue fazer e por quê.
2	**O modelo**	Autômato finito + pilha: a descrição informal e o papel do não-determinismo.
3	**Definição formal**	A 6-upla, a função de transição, a notação dos diagramas e o que é aceitar.
4	**Três exemplos do Sipser**	$\{0^n1^n\}$, $\{a^ib^jc^k \mid i = j \text{ ou } i = k\}$ e $\{ww^R\}$, com diagramas e rastreio da pilha.
5	**Fechamento**	AF versus AP, resumo e exercícios para a próxima aula.

Sipser, Introdução à Teoria da Computação, §2.2

MOTIVAÇÃO

Onde estamos

O que já sabemos

Autômatos finitos (determinísticos ou não) reconhecem exatamente as **linguagens regulares**.

A linguagem $\{0^n1^n \mid n \geq 0\}$ **não é regular** (lema do bombeamento, §1.4): um AF não consegue "contar" uma quantidade ilimitada de 0s.

Gramáticas livres-do-contexto (§2.1) **geram** linguagens como essa: $S \to 0S1 \mid \varepsilon$.

A PERGUNTA DE HOJE

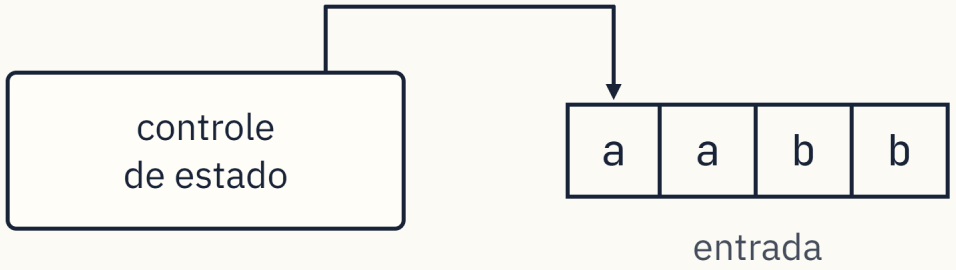
Que máquina *reconhece* as linguagens livres-do-contexto?

Resposta do Sipser: um autômato finito não-determinístico com uma memória extra — a **pilha**.

O MODELO

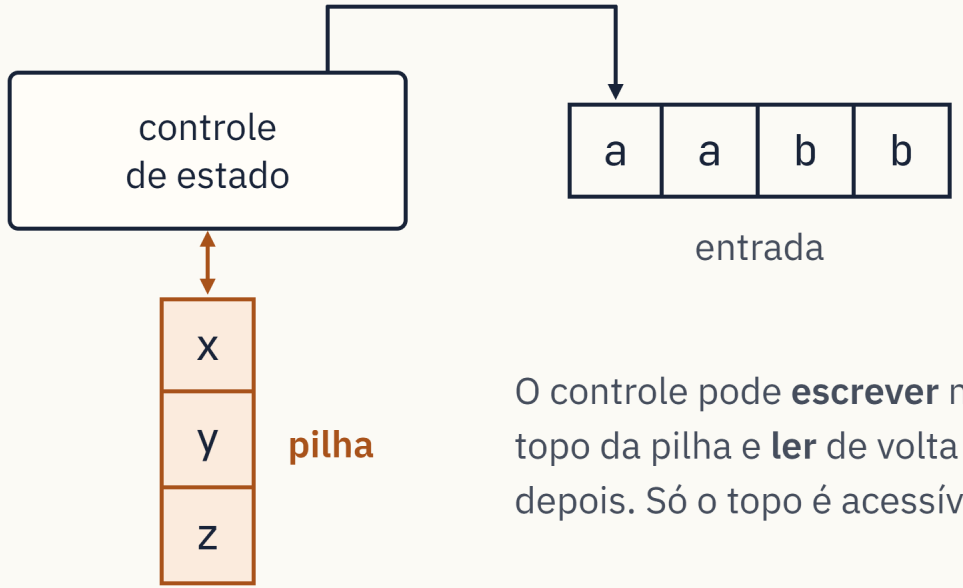
Um autômato finito com uma pilha acoplada

Autômato finito (Figura 2.11)



Controle = estados e função de transição. A cabeça só avança: cada símbolo é lido uma vez.

Autômato com pilha (Figura 2.12)



O controle pode **escrever** no topo da pilha e **ler** de volta depois. Só o topo é acessível.

Sipser, Introdução à Teoria da Computação, §2.2 — Figuras 2.11 e 2.12

O MODELO

A pilha: o último que entra é o primeiro que sai

empilhar (push)	Escreve um símbolo no topo. Os símbolos que já estavam lá são “empurrados para baixo”.
desempilhar (pop)	Lê e remove o símbolo do topo. O que estava abaixo volta a ser o topo.
só o topo	Todo acesso é pelo topo. Informação escrita “por baixo” fica inacessível até que o que está acima seja removido.
ilimitada	A pilha pode guardar uma quantidade ilimitada de símbolos — é isso que o AF não tem, e é o que permite reconhecer $\{0^n1^n\}$.

Como a pilha de pratos de uma cafeteria: o prato colocado por último é o primeiro a ser retirado.



Sipser, Introdução à Teoria da Computação, §2.2

O MODELO

Descrição informal: um AP para $\{0^n 1^n \mid n \geq 0\}$

> *"Leia os símbolos da entrada. À medida que cada 0 é lido, empilhe-o. Assim que os 1s forem vistos, desempilhe um 0 para cada 1 lido. Se a leitura da entrada for terminada exatamente quando a pilha fica vazia de 0s, aceite a entrada. Se a pilha ficar vazia enquanto 1s permanecem, ou se os 1s se acabam enquanto a pilha ainda contém 0s, ou se quaisquer 0s aparecem na entrada após os 1s, rejeite a entrada."*
>
> Sipser, §2.2, p. 115

`lê 0`	empilha um **0** (a pilha conta os 0s)
`lê 1`	desempilha um **0** (cada 1 "paga" um 0)
`fim`	entrada terminou **e** a pilha ficou vazia de 0s → **aceita**
`senão`	sobram 1s, sobram 0s ou aparece 0 depois de 1 → **rejeita**

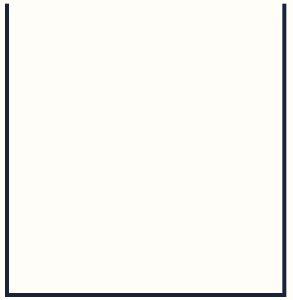
O MODELO

A pilha em ação na entrada 0011

INÍCIO

pilha vazia

`resta: 0011`



LÊ 0

empilha 0

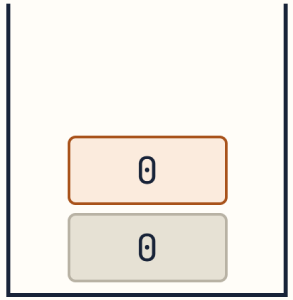
`resta: 011`



LÊ 0

empilha 0

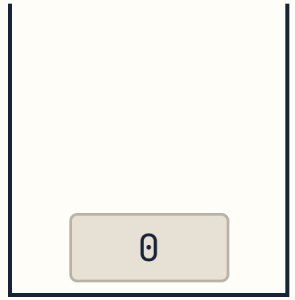
`resta: 11`



LÊ 1

desempilha 0

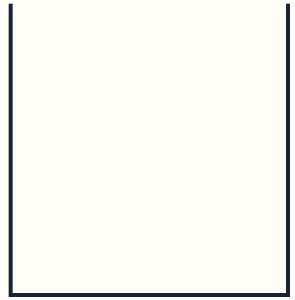
`resta: 1`



LÊ 1

desempilha 0

`resta: ε`



FIM

entrada acabou e pilha vazia

`resta: ε`

aceita

Sipser, Introdução à Teoria da Computação, §2.2

O MODELO

Autômatos com pilha são não-determinísticos

POR DEFINIÇÃO

A função de transição devolve um conjunto de possibilidades

Como num AFN, a máquina pode ter vários próximos movimentos legítimos — e transições com ε, sem ler a entrada.

DIFERENTE DOS AFS

APD e APN não são equivalentes em poder

AFN e AFD reconhecem a mesma classe (Teorema 1.39). Com pilha, não: há linguagens que só um AP não-determinístico reconhece (Sipser afirma sem provar).

NOSSA ESCOLHA

Nesta disciplina, “AP” significa AP não-determinístico

São eles que equivalem às gramáticas livres-do-contexto. Os Exemplos 2.16 e 2.18 exigem não-determinismo.

Lembrete: a máquina **aceita** se **algum** ramo da computação chega a um estado de aceitação ao final da entrada. Ramos que travam simplesmente morrem.

DEFINIÇÃO FORMAL · DEFINIÇÃO 2.13

Um autômato com pilha é uma 6-upla

$(Q, \Sigma, \Gamma, \delta, q_0, F)$

Q, Σ, Γ e F são todos conjuntos **finitos**. A pilha é ilimitada, mas o alfabeto que ela usa é finito.

Comparado ao AFN $(Q, \Sigma, \delta, q_0, F)$, só entra o alfabeto de pilha Γ — e muda o tipo de δ.

Q	conjunto de estados
Σ	alfabeto de entrada
Γ	alfabeto de pilha (pode ser diferente de Σ)
δ	função de transição: $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$
$q_0 \in Q$	estado inicial
$F \subseteq Q$	conjunto de estados de aceitação

Sipser, Introdução à Teoria da Computação, §2.2 — Definição 2.13

DEFINIÇÃO FORMAL

O coração do modelo: a função de transição δ

$$\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$$

DOMÍNIO — O QUE δ OLHA

o **estado atual**, o **próximo símbolo da entrada** (ou ε) e o **símbolo no topo da pilha** (ou ε)

CONTRADOMÍNIO — O QUE δ DECIDE

um **conjunto** de pares (**próximo estado**, **símbolo a empilhar** ou ε) — conjunto, logo não-determinismo

`ε na entrada`

a máquina se move **sem ler** nenhum símbolo da entrada

`ε no topo lido`

a máquina se move **sem olhar nem desempilhar** o topo da pilha

`ε no símbolo escrito`

a máquina **não empilha** nada nessa transição

`∅ como resultado`

nenhum movimento possível: esse **ramo trava** e morre

Sipser, Introdução à Teoria da Computação, §2.2 — $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$, $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$

DEFINIÇÃO FORMAL

Como ler os rótulos do diagrama de estados

a, b → c

a
lê a da entrada

b
desempilha b (exige b no topo)

c
empilha c no lugar

Rótulo	Lê da entrada	Desempilha	Empilha	Em português
a, b → c	a	b	c	lê a e troca o b do topo por c
a, ε → c	a	nada	c	lê a e empilha c (sem olhar o topo)
a, b → ε	a	b	nada	lê a e desempilha b
ε, b → c	nada	b	c	sem ler a entrada, troca b por c
ε, ε → $	nada	nada	$	empilha o marcador $ sem ler nada

Sipser, Introdução à Teoria da Computação, §2.2 — p. 117

EXEMPLO 2.14

M1 formalmente: o AP que reconhece $\{0^n1^n \mid n \geq 0\}$

ESTADOS

$Q = \{q1, q2, q3, q4\}$

ALFABETO DE ENTRADA

$\Sigma = \{0, 1\}$

ALFABETO DE PILHA

$\Gamma = \{0, \$\}$

INICIAL E ACEITAÇÃO

$q1$ e $F = \{q1, q4\}$

OBSERVE

O **$** está em Γ mas não em Σ: símbolo privado da pilha, o **marcador de fundo**.

q_1 aceita a cadeia vazia (n = 0); q_4 aceita as demais.

Falta δ — a tabela do próximo slide.

EXEMPLO 2.14

A função de transição δ de M1 em forma de tabela

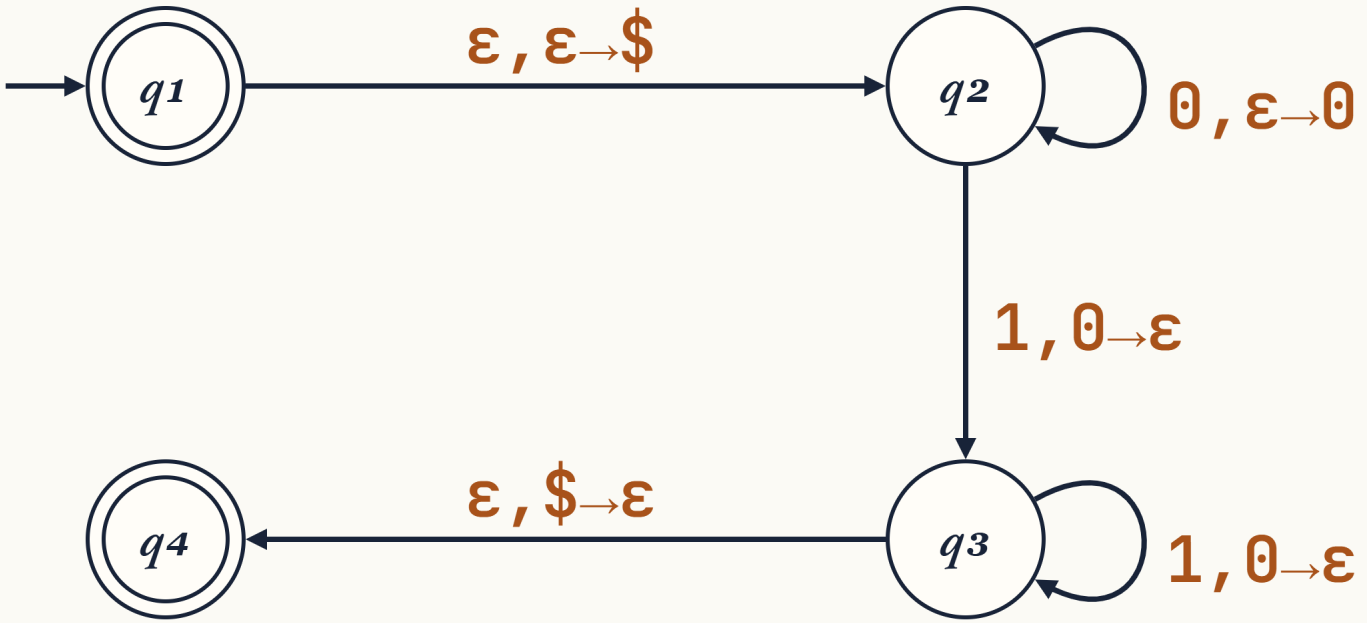
Entrada:	0			1			ε		
Pilha:	0	$	ε	0	$	ε	0	$	ε
q_1									$\{(q_2,\$)\}$
q_2			$\{(q_2,0)\}$	$\{(q_3,\varepsilon)\}$					
q_3				$\{(q_3,\varepsilon)\}$				$\{(q_4,\varepsilon)\}$	
q_4									

Entradas em branco significam **∅**: nenhum movimento possível. Das 36 entradas de δ, apenas 5 são não-vazias.

Leia $\delta(q_2, 1, 0) = \{(q_3, \varepsilon)\}$ como: em q_2, lendo 1 com 0 no topo, vá para q_3 e desempilhe (não escreva nada).

Sipser, Introdução à Teoria da Computação, §2.2 — Exemplo 2.14, p. 117

Diagrama de estados de M1 — reconhece $\{0^n1^n \mid n \geq 0\}$



LEGENDA

círculo duplo = aceitação
q_1 = inicial (seta solta)

Sipser, Introdução à Teoria da Computação, §2.2 — Figura 2.15

EXEMPLO 2.14

Rastreio de M_1 na entrada 0011

Passo	Estado	Lê	Transição usada	Pilha depois (topo à esquerda)	Entrada restante
0	q_1	—	início	ε (vazia)	0011
1	$q_1 \to q_2$	ε	ε, ε → $	$	0011
2	$q_2 \to q_2$	0	0, ε → 0	0 $	011
3	$q_2 \to q_2$	0	0, ε → 0	0 0 $	11
4	$q_2 \to q_3$	1	1, 0 → ε	0 $	1
5	$q_3 \to q_3$	1	1, 0 → ε	$	ε
6	$q_3 \to q_4$	ε	ε, $ → ε	ε (vazia)	ε

Entrada esgotada em $q_4 \in F \to$ **M_1 aceita 0011**. Na notação da definição: estados $q_1\ q_2\ q_2\ q_2\ q_3\ q_3\ q_4$ e pilhas ε, $, 0$, 00$, 0$, $, ε.

Em 001 a entrada acaba em $q_3 \notin F$: rejeita. Em 011, após o primeiro 1 a pilha é só $ e não há transição "1, $ → …": trava.

Sipser, Introdução à Teoria da Computação, §2.2 — Exemplo 2.14

DOIS TRUQUES DE PROJETO

O que a definição não dá, mas conseguimos simular

NÃO EXISTE

Testar se a pilha está vazia

δ só enxerga o símbolo do topo; “pilha vazia” não é um símbolo.

SOLUÇÃO

Empilhe um marcador especial **$** logo no início (ε, ε → $). Quando o $ reaparecer no topo, a pilha está “efetivamente vazia”. Nos diagramas, a transição final é sempre ε, $ → ε.

NÃO EXISTE

Testar se a entrada acabou

A máquina não vê um “fim de cadeia”; só lê símbolos ou ε.

SOLUÇÃO

Projete de modo que o estado de aceitação só seja alcançado **quando a entrada terminou** — a aceitação exige estar em F ao final. Em M_1, q_4 não tem saídas: se sobrar entrada, o ramo morre.

Daqui em diante, como o Sipser, assumimos que APs “podem testar” pilha vazia e fim da entrada, sabendo que é assim que se implementa.

EXEMPLO 2.16

Quando a máquina precisa adivinhar

A LINGUAGEM

$$\{a^i b^j c^k \mid i, j, k \geq 0 \text{ e } i = j \text{ ou } i = k\}$$

Exemplos: aabbc (i = j), aabccc (i = k), ε, abc. Fora: aabbbc, ba.

O AP lê e **empilha os a's**. Ao terminarem, tem todos na pilha para casá-los com os b's **ou** com os c's — mas não sabe antecipadamente com quais. O não-determinismo entra na hora certa: a máquina **adivinha**.

RAMO 1 — APOSTA EM I = J

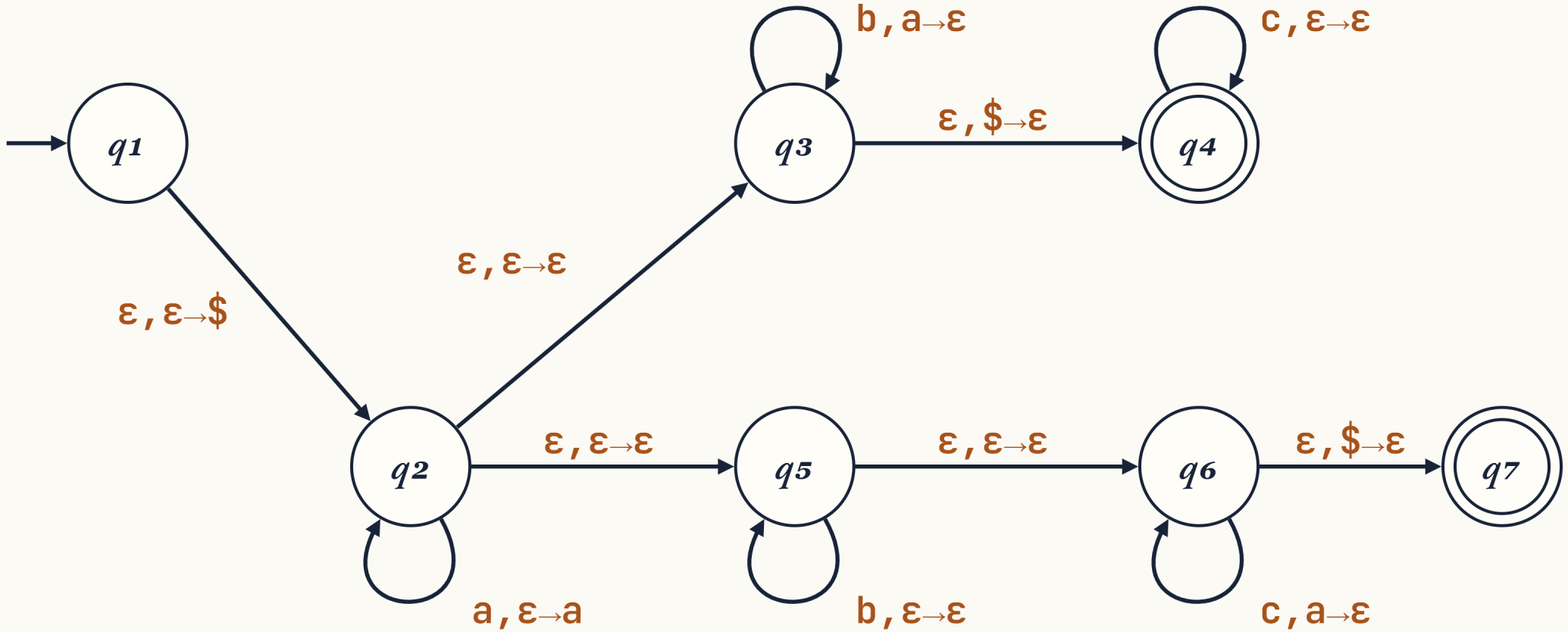
Desempilha um a para cada b lido; quando o $ aparece, lê os c's livremente e aceita.

RAMO 2 — APOSTA EM I = K

Lê os b's sem mexer na pilha; depois desempilha um a para cada c lido; aceita ao ver o $.

A máquina aceita se **algum** dos ramos aceita. Sipser: para essa linguagem o não-determinismo é **essencial** — nenhum AP determinístico a reconhece.

Diagrama de M2 — reconhece $\{a^i b^j c^k \mid i = j \text{ ou } i = k\}$



Sipser, Introdução à Teoria da Computação, §2.2 — Figura 2.17

Rastreio de M2 em aabcc (i = 2, j = 1, k = 2)

PREFIXO COMUM — $q_1 \to q_2$ com ε, ε → $; depois dois a's empilhados com a, ε → a. Situação: estado q_2, pilha **a a $**, restante **bcc**. Aqui a máquina se bifurca.

Ramo 1 (via q_3): aposta i = j

Estado	Transição	Pilha	Restante
$q_2 \to q_3$	ε, ε → ε	a a $	bcc
$q_3 \to q_3$	b, a → ε	a $	cc
q_3	nenhuma se aplica	a $	cc

Próximo símbolo é c, mas q_3 só lê b; e ε, $ → ε exige $ no topo (o topo é a). **Ramo trava: rejeita.**

Ramo 2 (via q_5): aposta i = k

Estado	Transição	Pilha	Restante
$q_2 \to q_5$	ε, ε → ε	a a $	bcc
$q_5 \to q_5$	b, ε → ε	a a $	cc
$q_5 \to q_6$	ε, ε → ε	a a $	cc
$q_6 \to q_6$	c, a → ε	a $	c
$q_6 \to q_6$	c, a → ε	$	ε
$q_6 \to q_7$	ε, $ → ε	ε	ε

Entrada esgotada em $q_7 \in F$. **Ramo aceita — logo M_2 aceita aabcc.**

Sipser, Introdução à Teoria da Computação, §2.2 — Exemplo 2.16

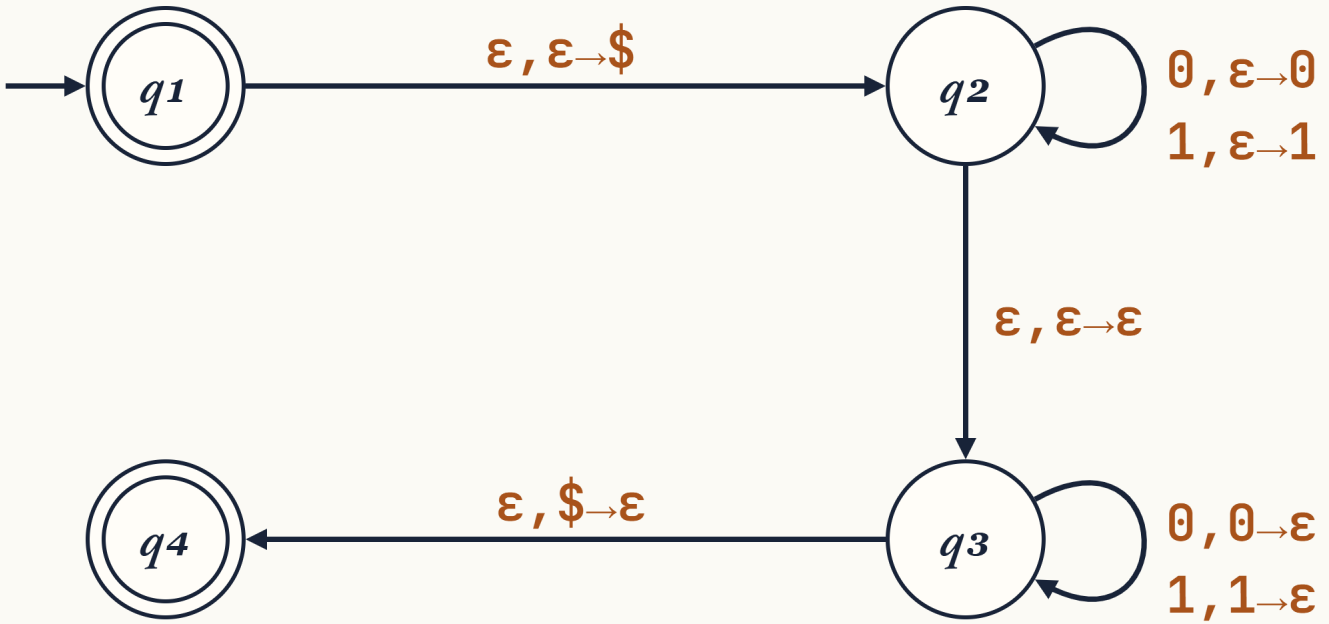
EXEMPLO 2.18

Palíndromos de comprimento par: $\{ww^R \mid w \in \{0,1\}^*\}$

> *"Comece empilhando os símbolos que são lidos. A cada ponto, adivinhe não-deterministicamente se o meio da cadeia foi atingido e, se tiver sido, passe a desempilhar um símbolo para cada símbolo lido, checando para garantir que eles sejam os mesmos. Se eles forem sempre os mesmos e a pilha esvaziar ao mesmo tempo em que a entrada terminar, aceite; caso contrário, rejeite."*
>
> Sipser, §2.2, p. 119 — w^R é w escrita de trás para a frente

`fase 1`	lê a primeira metade **empilhando** cada símbolo
`adivinha`	em algum ponto, **aposta** que chegou ao meio (transição ε)
`fase 2`	para cada símbolo lido, **desempilha** e exige que seja igual
`por quê?`	a pilha devolve a primeira metade **invertida** — exatamente w^R. LIFO faz o trabalho.

Diagrama de M3 — reconhece $\{ww^R \mid w \in \{0,1\}^*\}$



ONDE ESTÁ A APOSTA

A transição ε, ε → ε de q_2 para q_3 pode ser tomada a qualquer momento: cada momento é um ramo.

Sipser, Introdução à Teoria da Computação, §2.2 — Figura 2.19

EXEMPLO 2.18

Rastreio de M3 na entrada 0110 — o ramo que acerta o meio

Passo	Estado	Lê	Transição usada	Pilha depois (topo à esquerda)	Entrada restante
0	q_1	—	início	ε (vazia)	0110
1	$q_1 \to q_2$	ε	ε, ε → $	$	0110
2	$q_2 \to q_2$	0	0, ε → 0	0 $	110
3	$q_2 \to q_2$	1	1, ε → 1	1 0 $	10
4	$q_2 \to q_3$	ε	ε, ε → ε (aposta: aqui é o meio)	1 0 $	10
5	$q_3 \to q_3$	1	1, 1 → ε	0 $	0
6	$q_3 \to q_3$	0	0, 0 → ε	$	ε
7	$q_3 \to q_4$	ε	ε, $ → ε	ε (vazia)	ε

Entrada esgotada em $q_4 \in F \to$ **M_3 aceita 0110** ($w = 01$, $w^R = 10$).

Os outros ramos morrem: apostar o meio após “0” deixa q_3 lendo 1 com 0 no topo; após “011”, lendo 0 com 1 no topo. Só o palpite certo sobrevive.

Sipser, Introdução à Teoria da Computação, §2.2 — Exemplo 2.18

FECHAMENTO

Autômato finito versus autômato com pilha

	Autômato finito (cap. 1)	Autômato com pilha (§2.2)
Memória	só os estados (finita)	estados + pilha ilimitada (acesso pelo topo)
Tupla	$(Q, \Sigma, \delta, q_0, F)$	$(Q, \Sigma, \Gamma, \delta, q_0, F)$
Transição	$\delta : Q \times \Sigma\varepsilon \to \mathcal{P}(Q)$	$\delta : Q \times \Sigma\varepsilon \times \Gamma\varepsilon \to \mathcal{P}(Q \times \Gamma\varepsilon)$
Determinismo	AFD ≡ AFN em poder	APD ⊊ APN — não equivalentes
Linguagens	regulares	livres-do-contexto (próxima aula)
Exemplo típico	cadeias com número ímpar de 1s	$\{0^n1^n\}$, $\{ww^R\}$
Descrição equivalente	expressões regulares	gramáticas livres-do-contexto

Sipser, Introdução à Teoria da Computação, caps. 1 e 2

FECHAMENTO

O que levar desta aula

1

AP = AFN + pilha

Memória ilimitada, acessível só pelo topo (LIFO).

2

6-upla $(Q, \Sigma, \Gamma, \delta, q_0, F)$

$\delta : Q \times \Sigma\varepsilon \times \Gamma\varepsilon \to \mathcal{P}(Q \times \Gamma\varepsilon)$. Cada passo lê, desempilha e empilha — cada um opcional.

3

Rótulo "a, b → c"

Lê a, desempilha b, empilha c — qualquer um pode ser ε.

4

Marcador $ e aceitação no fim

Simulam o teste de pilha vazia e o de fim da entrada, que a definição não tem.

5

Não-determinismo é poder

Não é só conveniência: $\{a^i b^j c^k \mid i = j \text{ ou } i = k\}$ e $\{ww^R\}$ exigem adivinhar. Basta um ramo aceitar.

RECEITA DE PROJETO

Empilhe $ → fase de empilhar → aposta (ε) → fase de desempilhar casando símbolos → ε, $ → ε para aceitar.

Sipser, Introdução à Teoria da Computação, §2.2

EXERCÍCIOS · PARTE 1

Projete o AP: descrição informal e diagrama de estados

Para cada linguagem abaixo (Σ = {0,1} em (a) e (b)), dê uma descrição informal em português e o diagrama de estados, com os rótulos na notação "a, b → c". Indique onde entra o não-determinismo.

(a)

$\{w \mid w = w^R\}$

todos os palíndromos, de comprimento par **ou ímpar**. Dica: adapte M_3.

Sipser 2.4(e) + 2.5

(b)

$\{w \mid |w|$ é ímpar e o símbolo do meio é 0$\}$

Dica: a pilha não precisa guardar os símbolos, só **contar**.

Sipser 2.4(d) + 2.5

(c)

$\{a^i b^j c^k \mid i = j$ ou $j = k$, com $i, j, k \geq 0\}$

Parecida com o Exemplo 2.16, mas **não é igual**: o que muda nos dois ramos?

Sipser 2.10

Exercícios adaptados de Sipser, cap. 2

EXERCÍCIOS · PARTE 2

Descrições informais (Sipser, Exercício 2.7)

Dê descrições informais em português de APs para as linguagens abaixo. Diga o que a pilha guarda, quando empilha, quando desempilha e quando aceita.

(d)

cadeias sobre {a, b} com mais a's do que b's

Dica: a pilha pode funcionar como um **contador com sinal**.

Sipser 2.6(a) / 2.7(a)

(e)

o complemento de $\{a^n b^n \mid n \geq 0\}$ sobre {a, b}

Dica: de quantas maneiras uma cadeia pode **falhar** em ser $a^n b^n$?

Sipser 2.6(b) / 2.7(b)

(f)

$\{w\#x \mid w^R$ é subcadeia de x, com $w, x \in \{0,1\}^*\}$

Dica: combine a ideia de M_3 com uma **aposta** sobre onde w^R começa em x.

Sipser 2.6(c) / 2.7(c)

Exercícios de Sipser, cap. 2 — soluções de (d) e (f) no próprio livro, p. 138

PRÓXIMA AULA

Equivalência entre autômatos com pilha e gramáticas livres-do-contexto

TEOREMA 2.20

Uma linguagem é livre-do-contexto se e somente se algum autômato com pilha a reconhece.

PARA LER ANTES

Sipser, §2.2, “Equivalência com gramáticas livres-do-contexto”, p. 120–128.

PARA ENTREGAR

Os exercícios (a)–(f) desta aula, com diagramas para (a), (b) e (c).

REFERÊNCIA

SIPSER, M. *Introdução à Teoria da Computação*. Tradução da 2ª edição norte-americana. São Paulo: Cengage Learning. Cap. 2, §2.2.